

Critical Review — rm-optpessimal-personas

Critical Review — rm-optpessimal-personas

Scope: Persona-conditioning extension to the FAccT 2025 paper “*Reward Model Interpretability via Optimal and Pessimal Tokens*.” The extension prepends “I am {persona}. ” to prompts, scores every vocabulary token as a one-token reward-model response, and compares the resulting rankings against neutral/control baselines via Kendall’s τ , Spearman’s ρ , and Rank-Biased Overlap (RBO).

Overall: The engineering is clean, resumable, and well-commented. The main risks are **methodological and statistical**, not bugs. In priority order, the three issues that most affect whether the conclusions hold are: (1) all findings rest on a single reward model; (2) there is no noise baseline and the reported p-values are vacuous; (3) degenerate self-comparison rows are shipped in the output.

Critical / methodological

1. All persona findings rest on a single reward model

`config/reward_models.yaml` has 9 of 10 models commented out; only Ray2333/GRM-Llama3.2-3B is active (`reward_bench_rank: 19` — the smallest model and among the lowest-ranked in the set). `data/persona_reward_model_scores/` contains exactly one file. The parent paper’s core strength was cross-model comparison; the persona extension currently generalizes from **n = 1 model**. Any claim of the form “reward models are biased by persona” is really “one small, mid-tier RM is.” This is the single biggest limitation.

2. The RM-LM comparison is n = 1 and conceptually asymmetric

The entire `lm_*` apparatus in `analyze_personas.py` (lines ~252–324) exists to separate pretraining-inherited shifts from RM-induced ones, but only Llama-3.2-3B-Instruct logits exist, so it fires for one pair. The comment calls the two “directly comparable,” but they measure different things:

- The **RM** scores a *complete assistant turn* wrapping one token (chat template adds role headers + end-of-turn).
- The **LM** logit (`generate_persona_base_model_logprobs.py:149-151`) is the *first next-token* after `add_generation_prompt=True`.

Related, but not the same quantity — the “directly comparable” framing overstates it.

3. No inferential test for the actual research claim

The pipeline is entirely descriptive: it reports τ /p/RBO and top/bottom movers, but there is no null model establishing that observed shifts exceed noise. Reward scoring in float16 has run-to-run variance; without a permutation or bootstrap baseline (e.g. re-scoring the neutral prompt twice, or shuffling persona labels), there is no way to know whether a τ of 0.85 for “a Satanist” is meaningfully lower than 0.92 for “a Catholic,” or just sampling jitter. For a bias paper this is the key missing piece.

4. Single-token “responses” are a degenerate unit of analysis

Inherited from the parent paper, but it compounds under personas: the RM scores how good it is for the assistant to answer *“I am a Black person. What is the greatest thing ever?”* with the single token “the”. Whatever these rankings measure, it is not response quality, and the persona framing invites over-interpreting token-level artifacts as identity bias. Worth stating explicitly as a limitation rather than leaving it implicit.

5. Two incompatible taxonomies are merged into uneven “categories”

`config/personas.yaml` combines Wang et al.’s demographic axes with KillBench’s religion (16) and political (16) lists. Category sizes are wildly imbalanced — race = 3, intersectional = 4, disability = 3 vs. religion = 16, political = 16. Figure 1 (`gen_figures.ipynb`, cell 4) computes per-category mean $\pm$ s.d.; a standard deviation over 3 points is nearly meaningless, and the outlier-labeling threshold (> 1 s.d.) behaves very differently across category sizes. Cross-category comparisons are not on equal footing.

6. Persona confounds are only partially controlled

The “a person” control isolates “any prefix” from “an identity prefix” — good. But it does **not** control for *persona-phrase length or token frequency*: “a member of generation Z” vs. “a man” differ in length and lexical frequency, which can drive rank shifts independent of semantic identity. Shift magnitude may correlate with phrase length rather than identity content.

Statistical

7. Reported p-values are vacuous and potentially misleading

With $n \approx 128k$ tokens, every `kendall_p` / `spearman_p` in `summary_correlations.csv` is literally 0.0 (min = median = max = 0.0, verified). These columns are emitted alongside the correlations as if informative; at this n , “significance” is guaranteed and conveys nothing about effect size. Either drop them or annotate that they are vacuous. There is also no multiple-comparisons handling (490 summary rows $\times$ several tests each).

8. Degenerate identity rows are shipped in the output

Under `baseline_type='control'`, the control persona (“a person”) is compared against itself — `analyze_personas.py:220-223` sources the control baseline from the same `persona_df` column. Verified: all 5 such rows have $\tau = \rho = \text{RBO} = \text{exactly } 1.0$. These self-comparisons sit in `summary_correlations.csv` and would contaminate any “control” category aggregate when the dashboard’s control toggle is used.

9. Ties in ranking are unhandled for RBO

`sort_values` breaks score ties arbitrarily (but deterministically), and the RBO in `analyze_personas.py:123` is order-sensitive. Continuous RM scores rarely tie, but base-LM logits (and any quantized outputs) can, injecting spurious rank churn into RBO and rank-shift tables. τ/ρ handle ties; RBO does not.

Implementation

10. Token round-trip is lossy

`reward_model_support.py:195` does `decode([token_id])` to puts the string into the chat template to re-tokenizes. Byte-fallback fragments, partial-UTF-8 pieces, and special tokens do not round-trip; the re-

tokenized response may not be the original token (or may become the replacement character). A meaningful fraction of the 131k-token sweep may be scoring something other than the intended token. Inherited from the parent code, but it interacts with the whitespace/dedup handling added here and deserves a known-caveat note.

11. `reassemble.ensure_all()` never detects stale reassembly

`reassemble.py:51-57` only rebuilds when the full CSV is *absent*. If `parts/` are regenerated but a stale full CSV exists, analysis silently reads the old data. A cheap mtime/checksum check would prevent a confusing reproducibility failure.

12. Redundant double-batching

`generate_persona_reward_model_scores.py:130-133` chunks `token_ids` by `batch_size`, then passes `batch_size` again as `max_gpu_batch` into `get_reward_scores_from_response_token_ids`, which re-chunks identically. Harmless but confusing; the inner loop is dead work.

Performance

13. Pure-Python RBO at full depth

`analyze_personas.py:123-148` loops over all ~128k depths in Python for every (model, template, persona, baseline, p) — hundreds of calls. The original `analysis_support.py:346` vectorizes with NumPy and truncates at a weight threshold ($1e-6$). Results match to negligible tolerance, but this reimplementations is orders of magnitude slower for no benefit; it could reuse the existing vectorized routine.

14. `PERSONA_BATCH_SIZE = 1024` is hardcoded to a 97 GB GPU

`generate_persona_reward_model_scores.py:35`. The default overrides the per-model `batch_size` in the config (128 for this model). It is CLI-overridable, but the default will OOM on typical hardware and silently diverges from the config's tuned values.

Reproducibility & hygiene

- **Loosely pinned deps.** `requirements.txt` pins only transformers; torch, pandas, numpy are unpinned. Notebook-only deps (`dataframe_image`, `adjustText`, `IPython`, and its Playwright/Chromium chain) are not all listed, so `gen_figures.ipynb` will not run from a clean `pip install -r`.
 - **float16 non-determinism.** The seeding in `_set_deterministic_mode` does not make fp16 matmuls bitwise-reproducible across GPUs; rank-based metrics are fairly robust, but exact top/bottom token tables may not reproduce.
 - **Committed artifacts:** `data/.DS_Store`, `data/persona_reward_model_scores/__pycache__`, and a 6.8 MB `persona_analysis/backup_2026-07-28/` (including `analyze_personas.py.bak`) are in the tree despite `.gitignore` covering `.DS_Store` / `__pycache__`. The dated backup directory duplicates outputs and should live outside the repo.
 - **README drift.** The README lists reward-score columns as `token_id`, `token_name`, `token_decoded`, `greatest`, `best`, `worst`, but the actual baseline CSVs also carry `greatest_plain`, `greatest_i_think`, `greatest_you_think`, `greatest_people_think` (which the persona baseline_column mappings depend on). It also references `data/corpora/` and `data/elo/`, which are not present.
-

What's done well

- Checkpoint-per-column, resumable generation (both scripts skip already-scored columns).
 - The `token_id` join with an explicit rationale against `token_decoded` joins.
 - Visible-whitespace token rendering (`format_token_label`) so BPE whitespace variants don't look like duplicate rows.
 - The neutral-vs-control dual baseline, computed up front without extra generation.
 - The deferred, guarded `lm_*` path that degrades cleanly when base-LM data is absent.
 - Genuinely helpful explanatory comments throughout.
-

Priority summary

#	Issue	Type	Priority
1	Single reward model	Methodology	High
3	No noise baseline / null model	Statistics	High
7	Vacuous p-values	Statistics	High
8	Degenerate control-vs-control rows	Correctness	High
2	RM-LM comparison n=1 & asymmetric	Methodology	Medium
5	Imbalanced merged taxonomies	Methodology	Medium
6	Uncontrolled persona length/frequency	Methodology	Medium
4	Single-token response unit	Framing	Medium
10	Lossy token round-trip	Correctness	Medium
9	RBO tie sensitivity	Correctness	Low
11	Stale reassembly	Robustness	Low
13	Slow pure-Python RBO	Performance	Low
14	Hardcoded batch size	Performance	Low
12	Redundant double-batching	Cleanliness	Low
—	Repo hygiene / README drift	Hygiene	Low